

Conceptos importantes:

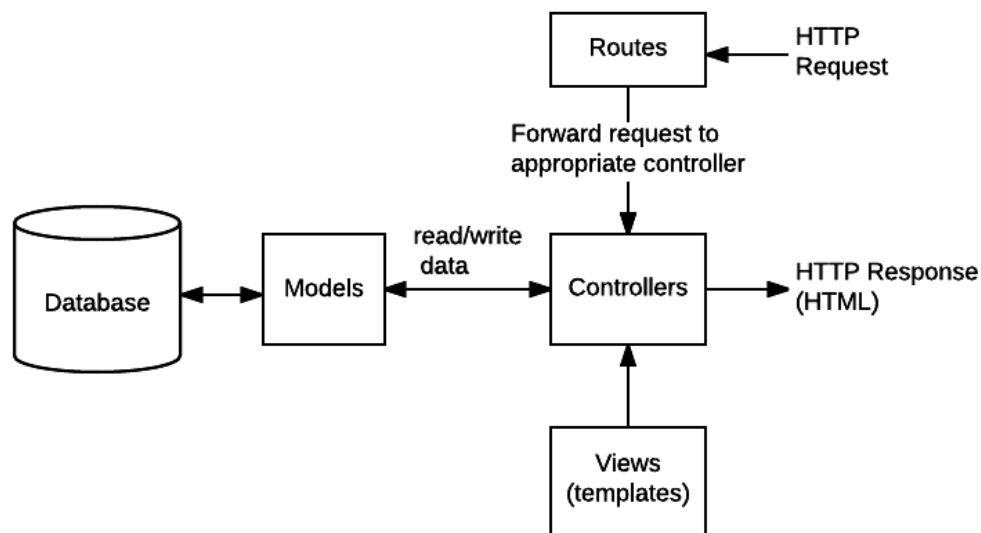
- **Protocolo http:** <https://www.youtube.com/watch?v=l2MihYAj0lw>
- **REST API:** <https://www.geeksforgeeks.org/node-js/rest-api-introduction/>
- **Monorepo (base de arquitectura):** <https://monorepo.tools/>, no es necesario leerse toda la pagina, solo la introducción para entender el concepto
- **Estructuración de carpetas:**
<https://medium.com/@jonoyanguren/mvc-pattern-in-nodejs-and-express-old-but-gold-46c21bee365a>, mas que nada tener en cuenta que es un controlador (controller), y la estructura general, no es necesario tener una carpeta de entities, con crear una carpeta por entidad ya es suficiente, ya que no estamos trabajando con una base de datos local, modelos como tal no se tendran, sino que se trabajara con tipos simplemente, segun investigue supabase puede generar un archivo con tipos de la base de datos: <https://supabase.com/docs/guides/api/rest/generating-types>. Para nuestros microservicios el servidor de express se levanta desde [index.ts](#), en la pagina se muestra como [server.ts](#).
- **Middleware:** Base de express, este es un concepto amplio dentro del framework, que trabaja con **Request**, **Response** y un **Next**, para efectos de nuestro desarrollo se le denomina al middleware como una función intermedia en caso de querer realizar una validación previa al controlador, por lo que, un middleware siempre debe terminar con un next (en nuestro contexto), de hecho, un controlador tambien es un middleware, pero que devuelve una response hacia el cliente (siempre termina con **Res**), por eso la división de los nombres.
<https://expressjs.com/en/guide/using-middleware.html>, En general dentro de la página de express está bien detallado todo lo más importante del framework

```
✓ backend
  > dist
  > node_modules
  ✓ src
    > core
    ✓ entities
      > area
      > auth
    ✓ reservation
      TS reservation.controller.ts
      TS reservation.model.ts
      TS reservation.routes.ts
    > user
    > vehicle
  > lib
  > scripts
  > tests
  TS server.ts
```

- **Routes:** <https://expressjs.com/en/guide/routing.html> el routing de express se basa en el protocolo http, y define el como desde las request a las URIs, también llamadas Endpoints, se realizan las responses, los endpoints son definidos con los métodos del protocolo HTTP: GET, POST, PATCH, UPDATE, DELETE, de entre los más comunes. La estructura de como es definido un endpoint es la siguiente:

METODO("uri complementaria a la del servidor: /login", middleware si requiere, controlador).

En palabras más generales se define hacia que URI el usuario quiere obtener, agregar, eliminar o modificar datos del sistema. Debe existir cierta lógica que realice el backend para aplicar lo que solicita el cliente, es aquí donde entra un middleware si requiere, por ejemplo validar el rol del usuario posterior a la validación continuar con NEXT() y pasar al controlador que realizar la lógica de negocios y termina con una Response hacia el cliente.



Nosotros como estamos con microservicios las rutas se definirían de la siguiente manera:

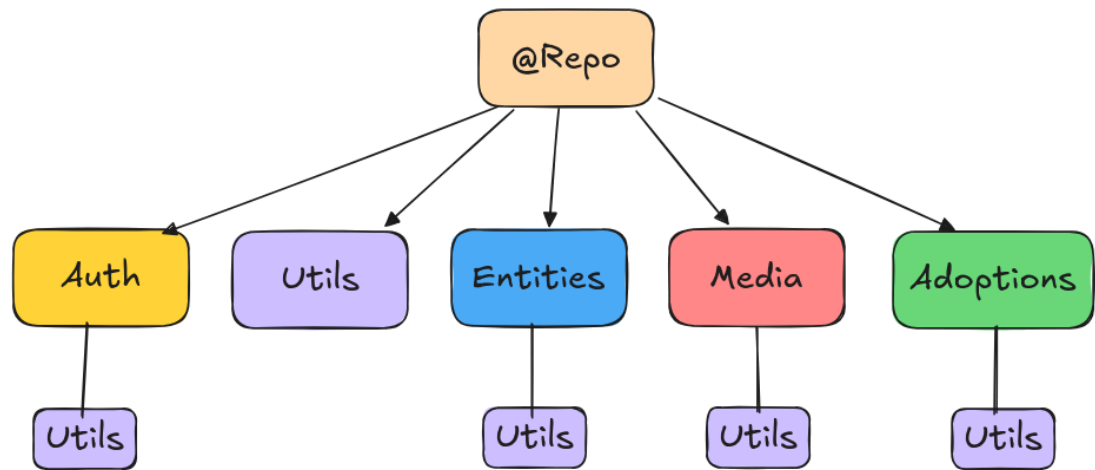
Dentro de [index.ts](#) en Auth microservice:

`app.use("/api/auth", router)` -> esto dice que el microservicio auth recibirá bajo /api/auth las rutas definidas en router, pudiendo ser /login, /register, conformado el endpoint: <http://ip:port/api/auth/login>, como estamos con nginx quedaria: <http://ip/api/auth/login>

Para otros casos microservicios donde hay múltiples entidades, las rutas se definen en función de eso, por ejemplo con el microservicio de Adoptions: ,

`app.use("api/posts", postsRouter)`
`app.use("api/messages", messagesRouter)`

- **Microservicios:** nuestro proyecto funciona en base al siguiente diagrama:



En donde **UTILS** representa un microservicio que actúa como librería para los demás microservicios, proporcionando la instancia del cliente de la bd, funciones de utilidades como de hasheo, tokens, verificaciones, tipos, etc. La utilización de esta se basa simplemente en crear la utilidad por ejemplo `verifyRUT()` y exportarla, luego dentro de cualquier microservicio importarla como:

```
Import { verifyRUT } from "@repo/Utils"
```

Y de esta manera se puede utilizar CUALQUIER utilidad creada dentro del microservicio de utils en otros microservicios.

Pero, esto no ocurre de la nada, se debe definir en el `package.json` la dependencia al microservicio de utils para poder importar (a modo de entendimiento de cómo funciona).

Ej:

```
Dependencies {  
  "@repo/Utils": "workspaces:*"  
}
```

Además dentro del `package.json` es donde se almacenan todas las librerías de ese microservicio, como acotación si en un futuro se requiere de instalar alguna librería, se navega hacia el directorio del microservicio, suponiendo que se está en la carpeta raíz del proyecto:

```
Cd apps/web -> npm install "nombre de paquete"
```

Librerías o tecnologías que si o si deben aplicarse y entenderse:

- **JSON:** Estándar de response definido en el REST API, es un formato de archivo ligero que funciona en formato key: value, nuestro backend está configurado para responder automáticamente con un json el cual lleva un status opcional, message, y values, ej:

```
{  
  "id": 1,  
  "name": "Tomás C.",  
  "email": "tcurihual2022@alu.uct.cl"  
}
```


En el front también debe aplicarse ya que es la estructura en la que las respuestas del backend son recibidas
- **JWT (back):** primordial entender, es un string UNICO en base a un campo de una entidad y un SECRET, este puede almacenar payloads (campos del usuario importantes, rol, id, etc) dentro del token en formato Json, la web tiene un espacio para mostrar el funcionamiento: <https://www.jwt.io/>
- **CORS (back principalmente):**
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>
- **BCRYPT (back):** usado principalmente para el hasheo y verificación de contraseñas (evitar guardar en texto plano en bd), <https://www.npmjs.com/package/bcryptjs>
- **Manejo de cookies (front)**

Opcionales:

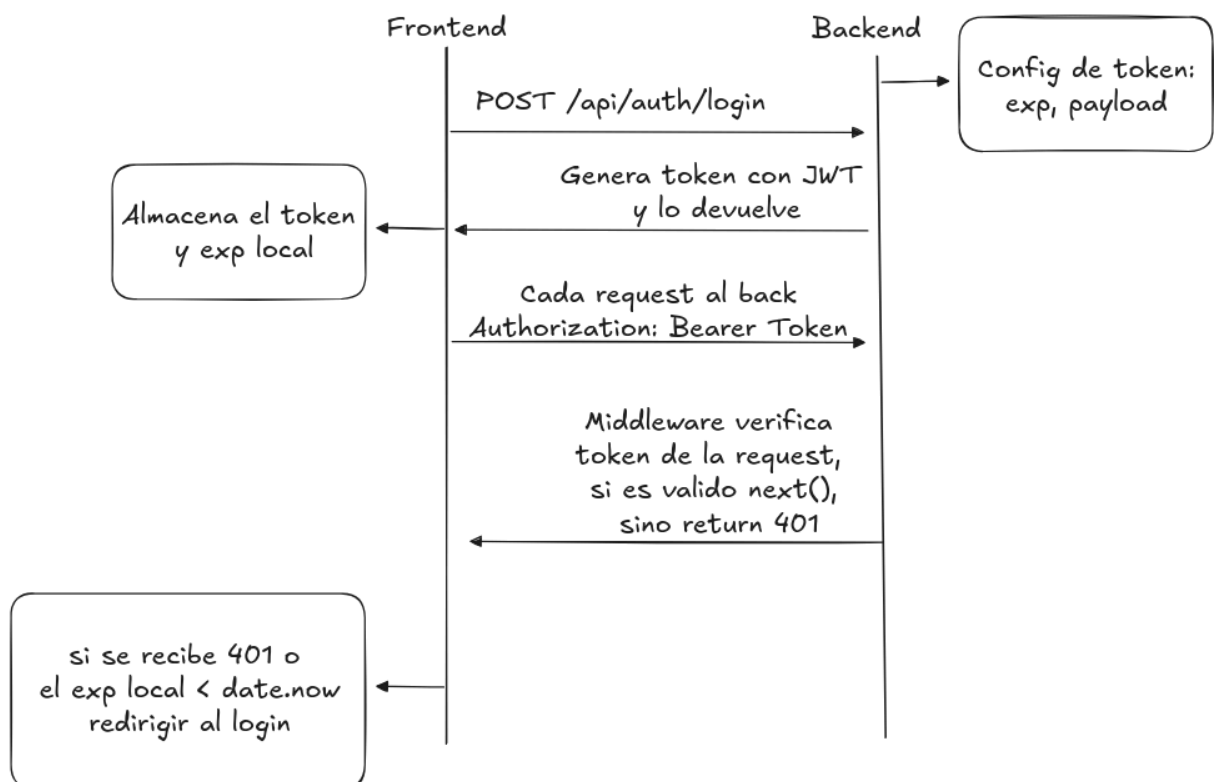
- **Contextos de React (front):** esto es un poco más complejo de entender, pero a grandes rasgos permite que TODA la app web pueda consumir por ejemplo funciones, estados, etc.
 - Por ejemplo si se crea un contexto de autenticación, se puede desde cualquier punto del sistema, acceder a un estado de autenticación, en base a eso permitir que el usuario pueda acceder a alguna vista o no (protección de rutas)
- **ZOD (front):** librería que utiliza esquemas para definir reglas de validación para entidades, previene que se le asigne valores a campos que no deberían llevar ese valor, permite asignar errores por campo, se usa en conjunto a react-hook-form
- **React-hook-form (front):** librería de formularios dinámico, en base a esquemas de zod, muestra errores automáticamente en caso de un campo erróneo, PREVIENE el envío del formulario si es que este está mal. Zod y react-hook-form, pueden dejarse para el siguiente sprint y aplicar tareas de refactorización cuando los integrantes ya están más familiarizados con el desarrollo

Objetivo sprint 1:

Los objetivos es consolidar el flujo de autenticación en el backend y en el frontend, creando los endpoints necesarios dentro del back y vistas propuestas durante sprint 0 como también desarrollo de nuevas, con prioridad en la implementación de la autenticación en el frontend

Back: implementar el flujo de autenticación, implicando:

- **Register:** endpoint
- **Login:** endpoint
- **Logout:** endpoint
- **validateRole:** middleware
- **Ownership:** middleware
- **validateSession:** middleware
- **Creación de utilidades en UTILS:** configuración de bcrypt (validar y hashear), configuración de JWT (config de los tokens (duración, uso de las .env, etc), verificar token, crear token), tipos



Para los endpoints se entiende que se debe considerar en las tareas que debe quedar ruteado, se pueden crear tareas iniciales para crear las rutas, etc.

Front: implementar vistas, contextos simples para modales, alertas, loading

- Implementar vistas contempladas en el figma, crear nuevas vistas, implementar estas nuevas.
- Contexto de modal: que permita abrir un modal en cualquier parte de la app, provee de un componente de modal que se puede cerrar, abrir, tiene destinos tamaños, etc y

permite que se ingrese cualquier componente dentro,
<https://www.uifrommars.com/que-es-un-modal/>

- *Más de lo mismo para los otros modales*
- *Los formularios puedes aplicarlos de forma vanilla o con las librerías recomendadas (zod, react-hook-form)*